Semantic Analysis in Compiscript

What the tree is, what the visitor does, and how to add a rule

September 2026

Contents

0. The one-paragraph version

You already have a **parser**: it turns source text into a *tree*. Semantic analysis is a second pass that **walks that tree** and asks the questions the grammar physically cannot ask — "was this variable declared?", "is `"hola" - 3` legal?", "does this `break` sit inside a loop?". The walk is implemented with the **visitor pattern**: one class (`SemanticChecker`) with one method per grammar rule. Each method inspects its node, may record an error, and — for expressions — **returns the node's type**. That return value is how type information flows *upward* through the tree. State that must flow *sideways* (the current scope, whether we're inside a loop, what the enclosing function must return) lives in instance fields on the checker.

Everything else in this document is detail on those two sentences.

1. The trip one line of code takes

Here's the whole thing as a walkthrough. Follow a single line — `let x: integer = "hola";` — from the editor to the red squiggle, naming what touches it and what each thing hands to the next.

You hit Run in the IDE. `frontend/` POSTs the file contents to `/api/run`. `src/server.py` does nothing interesting with it — it just reads the file off `workspace/` and calls into `src/compiler.py`. That file is the only place in the repo that knows the shape of the whole pipeline; both the CLI (`main.py`) and the server go through it so they can't drift apart.

`compiler.py` starts the lexer. `CompiscriptLexer` (generated, in `src/generated/`) chews the characters into tokens: `'let'`, `Identifier`, `':'`, `'integer'`, `'='`, `Literal`, `';'`. If you'd typed `@`, this is where it would die — nothing in `Compiscript.g4`'s lexer rules matches that character. Note that `compiler.py` first rips out ANTLR's default error listener and swaps in `src/error_listener.py`, which collects Spanish messages into a list instead of printing to stderr. Same listener gets attached to the parser a few lines later.

Then the parser. `parser.program()` takes that token stream and matches it against the parser rules in the grammar, and what falls out is the **parse tree** — a nested structure where every node corresponds to one grammar rule that matched (§2). Our line matches `variableDeclaration` cleanly, so no syntax error. Which is exactly the point: *the tree is structurally perfect and the program is still wrong.* The grammar said `typeAnnotation? initializer?` and both are present; it has no way to say they must agree.

Now the gate. `compiler.py` checks whether the error list is empty. If there were syntax errors it **stops here** and never runs the checker, because ANTLR error-recovers by inventing and skipping tokens — walking that half-fictional tree would bury the one real syntax error under twenty imaginary semantic ones. Clean parse, so we continue.

`SemanticChecker().check(tree).` This is the part you wrote. It's a subclass of the generated `CompiscriptVisitor`, and `check()` just calls `self.visit(tree)` and hands back the accumulated errors. The visit recurses down through `program → statement → variableDeclaration`, and at that last node **your `visitVariableDeclaration` fires** (§3 explains how the dispatch picks it).

Inside that method, three collaborators show up:

- It reads the annotation and calls `_resolve_type_node`, which turns the *parse node* for `integer` into a real `IntegerType()` object from `src/semantic/types.py`. Types are your own vocabulary, deliberately kept independent of the ANTLR classes — that's what makes them reusable for project 2.
- It builds a `Symbol` and hands it to `self.symbols.declare(...)` — `src/semantic/symbols.py`, the scope stack. That's the memory the grammar didn't have. It answers "is this name taken here?" (it returns `False` on a collision, which is how redeclaration gets caught) and later "what type is this name?".
- Then it visits the initializer subtree. That recursion bottoms out in `visitLiteralExpr`, which looks at `"hola"` and **returns `StringType()`**. That return value bubbles back up through the expression chain to us.

The actual check is one line: `value_type.is_assignable_to(symbol.type)` → `StringType` vs `IntegerType` → `False`. So `self._error(ctx, "...")`, which appends a `SemanticError` to the list in `src/semantic/errors.py` with the line and column pulled off `ctx.start`.

The walk finishes and two artifacts come out. The error list, and — this one's easy to miss — the **symbol table**. Every scope entered during the walk got linked into a permanent tree hanging off

`global_scope`, so after the walk it's all still there. `compiler.py` serialises it with `to_dict()`.

Back out to JSON. `compiler.py` returns errors + status message + `tree_json` + `symbol_table_json`; `server.py` ships it; the IDE renders the errors in the terminal pane, the parse tree in one panel and the scope tree in another.

That's the flow. Two files are generated and untouchable (`src/generated/`), one file is the grammar (`src/grammar/Compiscript.g4`), one orchestrates (`compiler.py`), and **four are yours**: `types.py`, `symbols.py`, `errors.py`, `checker.py`. Ninety percent of the work you'll ever do is in the last one.

2. Theory: what the tree actually is

2.1 Why a second pass is unavoidable

The lexer and parser only know *shape*. `let x: integer = "hola";` is perfectly grammatical. Nothing in a context-free grammar can express "the initializer's type must be compatible with the annotation", because that requires **remembering** something (what `x` was declared as) across an unbounded distance in the text. Grammars have no memory; a tree walk with a symbol table does. That gap is the entire reason semantic analysis exists.

2.2 The parse tree is literally the grammar rules, nested

Every rule you write in `Compiscript.g4` becomes:

- a **method** on the parser (`parser.program()`, `parser.block()`, …),
- a **context class** (`ProgramContext`, `BlockContext`, …), and
- a **node type** in the tree.

A node is an *instance* of the context class for the rule that matched at that point, and its children are the nodes and tokens that rule consumed, **in source order**. The tree isn't some abstract thing you have to imagine — it's a mechanical picture of "which rule matched what".

For `let x: integer = 5;`:

```
program
└── statement
    └── variableDeclaration                ← VariableDeclarationContext
        ├── 'let'                          ← TerminalNode (token)
        ├── x                              ← TerminalNode (Identifier)
        ├── typeAnnotation                 ← TypeAnnotationContext
        │   ├── ':'
        │   └── type
        │       └── baseType → 'integer'
        ├── initializer                    ← InitializerContext
        │   ├── '='
        │   └── expression
        │       └── ... → literalExpr → 5
        └── ';'
```

Note the long `expression → assignmentExpr → conditionalExpr → logicalOrExpr → … → literalExpr` chain for a bare `5`. That chain is **precedence made structural**: each level of the expression grammar is a level of binding tightness, so `1 + 2 * 3` parses with the multiplication *deeper* in the tree than the addition — which is exactly why evaluating bottom-up gives the right answer. The price is that even a trivial literal travels through ~8 nodes. This has a direct consequence for the checker, see §4.4.

2.3 Concrete syntax tree vs. AST

What ANTLR hands you is a **concrete syntax tree** (a.k.a. parse tree): it keeps *everything*, punctuation included — the `'('`, the `';'`, the `':'` are real child nodes. Many production compilers immediately convert this into a leaner **abstract syntax tree** that drops punctuation and collapses those precedence chains.

This project does **not** do that. It runs semantic analysis directly on the concrete tree. That's a normal choice for a course project — fewer moving parts — but it's why you see code like `ctx.getChild(2 * i - 1)` (`_operator_before`, `checker.py:112`) to fish out an operator token, and why `visitPrimaryExpr` (`checker.py:409`) has to hand-handle the `'(' expression ')'` case: the parentheses are actual children, and the default "return the last child's result" would return the `')'`.

2.4 Labeled alternatives

Compare:

```
assignment
  : Identifier '=' expression ';'
  | expression '.' Identifier '=' expression ';'
  ;

primaryAtom
  : Identifier                          # IdentifierExpr
  | 'new' Identifier '(' arguments? ')' # NewExpr
  | 'this'                              # ThisExpr
  ;
```

`assignment` has **unlabeled** alternatives → ANTLR generates *one* `AssignmentContext` for both, and you disambiguate at runtime. That's what `visitAssignment` (`checker.py:267`) does by counting `ctx.expression()`: one item = plain form, two = property form.

`primaryAtom` has `# Label` alternatives → a *separate* context class and a *separate visitor method* per alternative (`visitIdentifierExpr`, `visitNewExpr`, `visitThisExpr`). Much nicer: the dispatch disambiguates for you.

Practical takeaway: if a rule is awkward to check because you keep asking "which alternative was this?", the fix is to add `# Labels` in the `.g4` and re-run `make generate`. That regenerates the parser everyone depends on, so coordinate it with the team.

3. The visitor pattern

3.1 The problem it solves

You have ~40 node types and you want to run an operation over a tree of them. Two options:

- Put a `check()` method on every node class → but those classes are **generated**, so you'd lose your work on every `make generate`, and a second pass (TAC generation in project 2) would mean editing them all again.
- Put all the behaviour in **one external class with one method per node type**, and let each node say "call the method for *my* type". That's the visitor pattern. It moves the operation out of the type hierarchy, so you can add operations — checker, TAC generator, pretty-printer — without touching the nodes.

ANTLR generates the base class for you: `CompiscriptVisitor` (`src/generated/CompiscriptVisitor.py`), one `visitXxx` per rule, **every one defaulting to `return self.visitChildren(ctx)`** — "recurse into my children, do nothing myself".

3.2 How dispatch actually works

```
class SemanticChecker(CompiscriptVisitor):
    ...

checker = SemanticChecker()
checker.visit(tree)          # ← starts the walk
```

`self.visit(node)` calls `node.accept(self)`, and each generated context class implements `accept` as roughly:

```
class BlockContext(ParserRuleContext):
    def accept(self, visitor):
        return visitor.visitBlock(self)      # ← the node knows its own method
```

So **the node picks the method**. This is double dispatch, and it's why you never write a big `if isinstance(node, ...)` chain. Override `visitBlock` and yours runs; don't, and the inherited default runs and just recurses.

Three consequences worth internalising:

- **You only override what you care about.** Every rule you don't touch is still traversed for free. That's why the Fase 0 skeleton ran end-to-end while reporting zero errors.
- **The moment you override a method, you own the recursion.** If your `visitBlock` doesn't call `visitChildren` or `self.visit(child)`, the entire subtree under that block is *never visited* and every rule inside it silently stops firing. This is the single most common bug in this style of code. Look at `visitForStatement` (`checker.py:828`): it visits the init clause, the condition, the increment and the block, each explicitly, precisely because it overrode the default.
- **Visit each child exactly once.** `visitForeachStatement` (`checker.py:314`) visits `ctx.expression()` itself and then only `ctx.block()` — *not* `visitChildren` — otherwise the iterated expression gets walked twice and every error inside it is reported twice.

3.3 Visitor vs. Listener (and why this project chose Visitor)

ANTLR also generates a **Listener** (`CompiscriptListener.py`), driven by the walker through `enterXxx`/`exitXxx` callbacks. Listener methods return nothing, so to compute "the type of this expression" you'd push and pop results on a manual stack in the exit callbacks.

The Visitor's `visitXxx` **returns a value**, and this project builds everything on that: *the return value of visiting an expression node is its* `Type`. `docs/Arquitectura.md` records the decision explicitly.

- **Listener** = you get notified, you can't easily compute values. Good for side-effect passes (collect all declarations, pretty-print).
- **Visitor** = you control the recursion and get values back. Required when children's results compose into the parent's result — which is exactly what type checking is.

4. The four conventions this codebase runs on

Learn these and `checker.py` stops looking arbitrary.

4.1 Visiting an expression returns its `Type`

```
def visitLiteralExpr(self, ctx):
    ...
    return IntegerType()          # a leaf manufactures a type

def visitAdditiveExpr(self, ctx):
    result = self._visit_type(operands[0])       # ask the left child
    for i in range(1, len(operands)):
        rhs = self._visit_type(operands[i])      # ask the right child
        result = self._check_additive(ctx, op, result, rhs)   # combine + validate
    return result                                # hand it to my parent
```

Types flow **bottom-up**. Leaves (literals, identifiers) manufacture them; operators combine and validate; statements consume them and check constraints (`visitIfStatement` demands a `BooleanType`).

Statement-level methods return `None` — nobody's asking them for a type.

4.2 `ErrorType` is a poison pill that stops cascades

If `x` is undeclared, `visitIdentifierExpr` reports *one* error and returns `ErrorType()`. Then `x + 1` sees an `ErrorType` operand and returns `ErrorType()` **without reporting a second error**; `let y: string = x + 1;` sees `ErrorType` and stays quiet too. One real mistake produces one message instead of five. `ErrorType.is_assignable_to` returns `True` in both directions for the same reason.

So the standard shape of a check starts with:

```
if isinstance(operand, ErrorType):
    return ErrorType()          # already reported downstream — say nothing
```

4.3 State that can't flow bottom-up lives on `self`

Some questions can't be answered by a return value, because they're about *context*, not about a subtree. Four fields carry that (`checker.py:49`):

Field	Question it answers	Pushed / popped in
`self.symbols` (`SymbolTable`)	"what names are visible here?"	`enter_scope`/`exit_scope`
`_function_return_stack`	"what must `return` produce? am I even in a function?"	`visitFunctionDeclaration`
`_loop_depth`	"is `break`/`continue` legal here?"	`visitWhile/DoWhile/For/Foreach`
`_class_stack`	"what does `this` refer to?"	`visitClassDeclaration`

They're **stacks**, not scalars, because these constructs nest — a function inside a function, a loop inside a loop. And every push is paired with a `finally: pop()`, so an error mid-subtree can't desynchronise the state for the *rest of the file*.

`_chain_base` is the odd one out: it threads information **sideways** within a single expression. `leftHandSide: primaryAtom (suffixOp)*` means `obj.metodo(1).campo` is a flat list of suffixes, and each suffix needs the type of whatever came *before* it — which its own `ctx` can't see. So `visitLeftHandSide` (`checker.py:421`) stashes the running type in `self._chain_base` before visiting each suffix, and `visitCallExpr`, `visitIndexExpr` and `visitPropertyAccessExpr` read it as their very first statement, before visiting anything nested that could overwrite it.

4.4 Pass-through rules must pass the real type through

This one bites everybody. Because of the precedence chain (§2.2), a plain `"hola"` still routes through `logicalOrExpr`, `logicalAndExpr`, `equalityExpr`, `relationalExpr`… So:

```
result = self._visit_type(operands[0])
if len(operands) == 1:
    return result            # ← no '||' actually present: I'm a wire, not a check
```

If `_check_logical` returned `BooleanType()` unconditionally, **every expression in the language** would type as boolean. Rule: when the optional operator isn't actually present, the method is a wire. Same pattern in `visitEqualityExpr`, `visitRelationalExpr`, `visitTernaryExpr`.

5. The four files that are yours

5.1 `types.py` — the type vocabulary

A small class hierarchy: `IntegerType`, `FloatType`, `BooleanType`, `StringType`, `NullType`, `VoidType`, three structural ones — `ArrayType(element)`, `FunctionType(params, ret)`, `ClassType(name, parent, members)` — and two special ones:

- **`UnknownType`** — "not decided yet". Produced by `let x;` (no annotation, no initializer), by an untyped parameter, and by `[]`. The first assignment **narrows it permanently**. It accepts anything, so it never generates errors on its own.
- **`ErrorType`** — "already broken, already reported" (§4.2).

The entire policy of the language lives in one method, `is_assignable_to(target)` — "can a value of *this* type flow into a slot declared as `target`?". The overrides are the design decisions from `docs/Arquitectura.md`, made executable:

- `IntegerType` → also assignable to `FloatType` (promotion, one direction).
- `NullType` → assignable to arrays and classes, never to primitives.
- `ClassType` → assignable to any ancestor (`is_subclass_of`).
- `ArrayType` → invariant, element types must match exactly.
- `ErrorType` / `UnknownType` → accept everything.

Because it's centralised, arguments, returns, assignments, `case` labels and ternary branches all share one consistent notion of compatibility. If the rules ever change, that's the one place.

5.2 `symbols.py` — the memory

`Symbol` = name + kind (`VARIABLE`/`CONSTANT`/`PARAMETER`/`FUNCTION`/`CLASS`) + type + line/column, plus an `address` field that's reserved and unused until project 2 needs it for code generation.

`Scope` = a `dict[str, Symbol]` + a `parent` pointer + a `children` list. The two directions are different things and both matter:

- **`parent` (upward)** is what name resolution walks. `resolve(name)` checks this scope, then its parent, then its parent… up to global. That one loop *is* lexical scoping — and it's why closures work with zero extra code: a nested function's scope chains up through the enclosing function's, so outer locals just resolve.
- **`children` (downward)** makes it a permanent **tree**. The `SymbolTable._stack` is transient — "what's open right now" — and gets popped as the walk leaves a construct, but nothing is destroyed, because the child is still linked under its parent. After the walk the whole tree is still reachable from `global_scope`, which is what feeds the IDE panel and what project 2 will use to lay out activation records.

`resolve_local` (this scope only, no parent walk) exists for two cases: **redeclaration** checks — "is this name taken *here*?", since shadowing an outer name is legal — and **class-member** lookup, since `obj.campo` must not fall through to some unrelated global named `campo`.

5.3 `errors.py` — the output

`SemanticError(line, column, message)` with a Spanish `__str__`, collected in a flat `SemanticErrorList`. Deliberately the same shape as `error_listener.py`'s lexical/syntax errors, so `compiler.py` can just concatenate the two lists for display.

5.4 `checker.py` — everything else

One class, ~50 methods, sectioned by owner. This is where all the actual rules live and where essentially all your future edits go.

6. What you actually do when you add a rule

Told the same way as the walkthrough: here's the order you go in.

Start in the grammar, but expect to read, not write. Open `src/grammar/Compiscript.g4` and find the rule that produces the thing you're checking. Read the right-hand side carefully, because it tells you exactly what accessors you'll have on `ctx`: one `Identifier` in the rule means `ctx.Identifier()` gives you a token, two or a `*` means it gives you a *list*. Check whether the alternatives are labeled (§2.4) — that decides whether you get a dedicated `visit` method or have to disambiguate by counting children. Most semantic rules need **no grammar change at all**. If you do need one (adding `# Labels`, say), that's a `make generate`, and it regenerates the parser the whole team builds on, so flag it.

Then check whether your vocabulary can express the answer. Can the existing `Type` classes say what you need? Usually yes. If you need a new compatibility rule, it goes in `is_assignable_to` in `types.py`, *not* scattered into the checker — that's how it stays consistent across arguments, returns, assignments and case labels. If you need to track a new kind of name, that's a `SymbolKind` in `symbols.py`.

Then decide if the rule needs context the subtree doesn't have. "Is `break` legal here?" isn't answerable from the `break` node — you need to know whether some ancestor is a loop. Anything like that becomes a field on the checker, pushed and popped in a `try/finally` by whichever `visit` method owns the construct (§4.3). Get this decision right up front; it's annoying to retrofit.

Now open `checker.py` and find your method. If it's already overridden — likely, most are — you're extending someone's existing logic, and the recursion is already wired. **Do not add a second walk of the same subtree**; fold your check into what's there. If it isn't overridden yet, add it in the right section, and remember that from that moment you own the recursion for that node.

Write the check itself, in this order: read the children off `ctx`, recurse where you need a child's type (via `_visit_type`, never raw `self.visit`, so an unimplemented rule returning `None` can't crash you), bail out early and silently if any operand is `ErrorType`, then compare and call `self._error(ctx, "mensaje en español")` on violation. Finish by returning the right thing: an expression's `Type`, or `None` for a statement.

Then make sure everything still gets walked. Trace your method: does every child that contains code get visited exactly once? Missing one means silent dead zones; doubling one means duplicate error messages.

Finally, a fixture and a test. Drop a `.cps` under `src/tests/semantic/<category>/` — one that trips the rule and one that shouldn't — assert on the messages, run `make test`.

The condensed checklist

- ☐ Read the `.g4` line first, always. What accessors does it give me?
- ☐ Labeled or unlabeled alternatives?
- ☐ Can `types.py` / `symbols.py` already express this?
- ☐ Does it need context state? → field on `self`, `try/finally`.
- ☐ Is the method already overridden? → extend, don't re-walk.
- ☐ Guarded against `ErrorType` / `None` so I don't cascade?
- ☐ Every child walked exactly once?
- ☐ Returning a `Type` (expression) or `None` (statement)?
- ☐ Fixture + test.

7. Worked example: reading `visitForeachStatement`

Grammar: `foreachStatement: 'foreach' '(' Identifier 'in' expression ')' block;`

Rules to enforce: the iterated expression must be an array; the loop variable is implicitly declared with the array's *element* type; it lives only inside the body; `break`/`continue` must be legal in there.

```
def visitForeachStatement(self, ctx):
    name = ctx.Identifier().getText()                    # (1)

    iterated_type = self.visit(ctx.expression())         # (2)

    if iterated_type is None:                            # (3)
        element_type = UnknownType()
    elif isinstance(iterated_type, ArrayType):
        element_type = iterated_type.element             # (4)
    elif isinstance(iterated_type, ErrorType):
        element_type = ErrorType()                       # (5)
    else:
        self._error(ctx, "la expresión de 'foreach' debe ser un arreglo")
        element_type = ErrorType()

    self.symbols.enter_scope(ScopeKind.BLOCK)            # (6)
    try:
        self.symbols.declare(Symbol(name, SymbolKind.VARIABLE,
                                    element_type, ctx.start.line, ctx.start.column))
        self._loop_depth += 1                            # (7)
        try:
            return self.visit(ctx.block())               # (8)
        finally:
            self._loop_depth -= 1
    finally:
        self.symbols.exit_scope()
```

1. **Read the node.** Grammar rule → accessor. One `Identifier` in the rule, so `ctx.Identifier()` is a single token.
2. **Recurse deliberately**, not via `visitChildren`, because we need the result *and* must not visit `expression` twice.
3. Defensive: a not-yet-implemented sub-rule returns `None`. Stay silent rather than emit a false error.
4. **The actual semantic rule**, three lines: array → element type.
5. **Don't cascade** (§4.2).
6. **Scope discipline**: a new scope so the loop variable dies at the closing brace; `try/finally` so an error inside can't leave the stack unbalanced.
7. **Context state**: bump `_loop_depth` so a `break` inside the body sees a nonzero depth and stays quiet.
8. **Own your recursion**: explicitly walk the body.

Every rule in the file is some subset of those eight moves. Declaration rules add "build a `Symbol`, and report if `declare()` returns `False`"; expression rules add "combine the child types and return one".

8. Worked example: adding one from scratch

Scenario: disallow division by a literal zero.

Following §6: the node is `multiplicativeExpr: unaryExpr (('*'|'/'|'%') unaryExpr)*`. Unlabeled and there's no dedicated method per operator, so I need the operator text at runtime — `_operator_before(ctx, i)` already does that, so no grammar change and no `make generate`.

No new type and no new symbol kind: the result type of a division is unchanged, this is purely a diagnostic. No context state either — everything I need is right here in the subtree.

`visitMultiplicativeExpr` (`checker.py:500`) is already overridden, so I'm extending its existing loop, not writing a new walk:

```
if op == "/" and operands[i].getText() == "0":
    self._error(ctx, "no se puede dividir entre cero")
```

`getText()` on a context returns the source text of that whole subtree — cheap and fine for a literal check like this. I keep returning the normal arithmetic result type rather than poisoning with `ErrorType`, because the expression is still well-typed; it's just doomed at runtime.

Then `src/tests/semantic/tipos/div_cero.cps` with a `10 / 0;` and a `10 / x;`, assert the first errors and the second doesn't, `make test`.

9. Gotchas actually present in this codebase

Real, documented traps in `checker.py`. They will confuse you if you meet them cold.

`ctx.type_()`, not `ctx.type()`. `type` is a Python builtin, so ANTLR's Python target appends an underscore. Same for any rule named after a keyword.

Every literal lexes as `Literal`. The grammar declares `Literal: IntegerLiteral | FloatLiteral | StringLiteral;` *before* the three specific rules, and ANTLR's earliest-rule tiebreak makes the specific token types effectively unreachable. So `visitLiteralExpr` (`checker.py:382`) sniffs the *text* — `startswith('"')` → string, contains `.` → float, else integer — instead of the token type. Not a hack for its own sake; a consequence of lexer rule ordering.

Unlabeled alternatives share one context class. `visitAssignment` has to count `ctx.expression()` (1 = `x = e;`, 2 = `o.p = e;`), and `visitClassDeclaration` has to index `ctx.Identifier()` positionally (`[0]` = class name, `[1]` = parent name).

Assignment is implemented twice. `x = 5;` matches the `statement → assignment` rule and goes through `visitAssignment`. But `arr[0] = 5;` matches *neither* of that rule's alternatives, so it falls through to `expressionStatement` and lands in `visitAssignExpr`. Two methods, and both have to handle `UnknownType` narrowing identically.

`_visit_type` instead of raw `self.visit`. An unimplemented rule falls through to the default `visitChildren`, which returns `None` — and `None` has no `.is_assignable_to`. `_visit_type` (`checker.py:94`) maps `None` to `ErrorType` so a half-finished checker degrades quietly instead of crashing. Scaffolding from the phased build, still a useful guard.

Class members alias the scope's dict, they aren't a copy. `class_type.members = self.symbols.current.symbols` binds the *same object*. That's what makes `this.nombre` resolve *while the class body is still being visited* — a snapshot taken at the end would be empty at that point.

Declaration happens before the initializer is walked. In `visitVariableDeclaration` the symbol is declared first, so `let x = x + 1;` resolves the right-hand `x` to the new declaration instead of erroring "undeclared". Whether that should be a "used before initialised" error instead is flagged as an open question in the code.

`_resolve_type_node` doesn't validate class names. Any non-primitive name becomes a `ClassType` on trust, declared or not — there's a live `TODO` at `checker.py:73`. So `let x: Foo;` for a nonexistent `Foo` currently passes silently.

10. Glossary

Token — a lexeme classified by the lexer (`Identifier`, `Literal`, `'let'`).

Parse tree / concrete syntax tree — the full derivation, punctuation included. What ANTLR gives you and what this project walks.

AST — a cleaned-up tree with punctuation and precedence chains removed. Not built here.

Context class — the generated node class for one grammar rule (`BlockContext`). Its accessors mirror the rule's right-hand side.

`ctx.start` — the first token of the node; source of the `line`/`column` on every error message.

Terminal node — a leaf holding a token.

Visitor / double dispatch — an external operation over the node hierarchy; `visit(node)` → `node.accept(visitor)` → `visitor.visitBlock(node)`.

`visitChildren` — the inherited default: recurse into all children and return the last one's result. Fine for pass-through rules, wrong the moment you need a value or a specific order.

Scope — one lexical name table plus a parent link.

Lexical scoping — resolution walks *enclosing source text*, not the call stack; implemented by `Scope.resolve`'s parent loop.

Shadowing — an inner declaration hiding an outer one of the same name. Legal, and it's why redeclaration checks use `resolve_local`, not `resolve`.

Symbol table — the stack of currently open scopes during the walk, plus the permanent scope tree it builds.

Type promotion / widening — `integer` flowing into a `float` slot.

Assignability — the compatibility relation; one method, `Type.is_assignable_to`.

Error recovery — the parser's ability to continue past a syntax error by inventing or skipping tokens. The reason semantic analysis is gated on a clean parse.

Dead code detection — the check in `visitBlock` that flags statements after a `return`/`break`/`continue` in the same block.